

Advanced Reading in Computer Vision (MAT3563)

LAB 03. CNN FOR VIDEO CLASSIFICATION

Phần này chúng ta thực hiện việc nhận dạng hành động (của con người) có trong một đoạn video: Đầu vào là một đoạn video – tức là chuỗi khung hình F_t phụ thuộc thời gian t . Mỗi khung hình F_t là một ảnh phẳng. Xét riêng từng kênh màu, toàn bộ đoạn video có thể coi là một dạng dữ liệu 3D. Đầu ra là phân loại hành vi của một cá thể có trong video. Bộ dữ liệu huấn luyện và kiểm tra được dùng trong phần này là UCF50 ([link https://www.crcv.ucf.edu/data/UCF50.rar](https://www.crcv.ucf.edu/data/UCF50.rar)), với 50 nhãn hành động. Các bạn có thể mở rộng để sử dụng những tập dữ liệu lớn hơn.

Phương pháp có thể mở rộng một cách cơ học cho các ứng dụng trong thị giác máy tính như:

- Hỗ trợ chẩn đoán dựa trên ảnh siêu âm hoặc chụp cắt lớp (đều là các dạng dữ liệu có thể biểu diễn dưới dạng một chuỗi ảnh 2D)
- Phân tích hành vi đám đông từ Video
- Một số bài toán dự đoán cho các luồng chuyển động .v.v.

Một số hướng tiếp cận cho bài toán này bao gồm:

- **Sử dụng mạng tích chập 3D** (3D convolution neural network): Mạng được xây dựng với ý tưởng tương tự như trường hợp 2D, Tuy nhiên khối lượng tính toán bây giờ sẽ lớn hơn hẳn. Chính vì điều này thông thường người ta sẽ bỏ bớt các frame. Từ cách chọn frame, có một vài dạng hướng tiếp cận đã được đưa ra để cải thiện hiệu quả của các mô hình dựa trên tích chập 3D:
 - o Sử dụng tích chập 3D với phương pháp chọn frame hợp lý
 - o Phương pháp xây dựng 2 đường (Two paths), trong đó tỉ lệ chọn Frame trên các đường là khác nhau (ví dụ một đường chọn theo tỉ lệ 1/8 – đường còn lại chọn theo tỉ lệ 1/32). Hướng này còn gọi là Fast – Slow paths. Trên mỗi hướng chúng ta vẫn sử dụng tích chập 3D, nhưng sau đó gộp kết quả lại. Thực nghiệm cho thấy hiệu quả tăng lên đáng kể (độ chính xác tăng nhưng khối lượng tính toán giảm).
 - **Sử dụng mạng tích chập 3D - Fast – Slow Paths** : Bản chất vẫn sử dụng mạng 3D, tuy nhiên chia làm hai nhánh. Một nhánh sử dụng nhiều frame hơn và nhánh còn lại ít hơn. Số frame được chọn sao cho tổng số frame được xử lý vẫn sẽ ít hơn so với số frame trong video gốc.
 - **Dựa trên mạng tích chập 2D**:
 - o **Kết hợp lớp tổng hợp (Fusion)**: Để ý khi sử dụng CNN cho bài toán phân loại ảnh màu, ở tầng đầu tiên chúng ta sẽ tổng hợp đầu ra ứng với các kênh màu thành 01 map đầu ra. Chúng ta sẽ áp dụng kỹ thuật tương tự trong tầng Fusion: Tức là xét các ảnh phẳng giống như trường hợp các kênh màu, ta sử dụng phép nhân chập 2D sau đó tổng hợp (cộng) kết quả lại. Ta có hai cách thực hiện tổng hợp:
 - **Early Fusion**: Tổng hợp từ tầng đầu – Tương tự như tổng hợp đầu ra sau tích chập của các kênh màu trong phân loại ảnh màu.
 - **Late Fusion**: Tổng hợp từ các tầng cuối.
- Nhược điểm của hướng tiếp cận này là thông tin về thứ tự xuất hiện của các frame sẽ bị mất. Tuy nhiên thực nghiệm cho thấy kết quả vẫn chấp nhận được.
- o **Kết hợp 2D CNN để trích xuất đặc trưng của mỗi lớp ảnh và sử dụng LSTM/RNN để xử lý chuỗi vector đặc trưng thu được**: Kiến trúc sẽ gồm phần CNN 2D áp dụng cho từng frame, từ đó tạo ra một chuỗi feature vector (phụ thuộc thời gian) và sử dụng LSTM dạng Many – One để xử lý bài toán phân loại với đầu vào là dữ liệu dạng chuỗi.
 - o **Two Streams**: Kiến trúc này tạo thành 02 luồng vào: Luồng thứ nhất sử dụng CNN trích xuất đặc trưng; Luồng thứ hai tính toán cái gọi là luồng dịch chuyển trong ảnh – Optical Flow.
 - o
- **Ngoài ra còn một số hướng tiếp cận khác các bạn có thể tự tìm hiểu.**

Trong bài thực hành này chúng ta chỉ tham khảo một số hướng như 3DCNN; Fussion 2D CNN; 2D CNN + LSTM.

A. SIMPLE 3D CNN MODEL FROM TENSORFLOW.KERAS

Có thể tham khảo 02 chương trình nguồn:

- Sử dụng mô hình pretrained 3D Resnet để dự đoán: `action_recognition_resnet3D.ipynb`
- Xây dựng một mô hình 3D đơn giản và huấn luyện + dự đoán: `Simple_CNN3D_action_recognition.ipynb`

Phần sử dụng mô hình pretrained các bạn tự tham khảo code do chúng ta gọi thẳng đến thư viện mô hình. Dưới đây chúng ta xem xét chi tiết hơn về cách xây dựng một mô hình 3D CNN đơn giản từ các Layer của thư viện và cách huấn luyện nó.

Các phần code thực nghiệm cho mô hình này được xây dựng trên TensorFlow – Keras được xây dựng trên version thấp của các thư viện này (TF 1.0 và 2.0), nên muốn thực nghiệm trên version cao hơn có thể phát sinh lỗi và cần thay đổi các thư viện này hoặc sửa lại đối tượng Config.

Dữ liệu phục vụ bài thực hành này là tập dữ liệu UCF50, các bạn lấy tại địa chỉ: <https://www.crcv.ucf.edu/data/UCF50.rar>.

Dữ liệu gồm một số mẫu video, phân thành 50 loại hành vi khác nhau, chia làm 25 nhóm hành vi. Mỗi nhóm hành vi có ít nhất 4 mẫu video. Phân loại các hành vi bao gồm:

Baseball Pitch, Basketball Shooting, Bench Press, Biking, Biking, Billiards Shot, Breaststroke, Clean and Jerk, Diving, Drumming, Fencing, Golf Swing, Playing Guitar, High Jump, Horse Race, Horse Riding, Hula Hoop, Javelin Throw, Juggling Balls, Jump Rope, Jumping Jack, Kayaking, Lunges, Military Parade, Mixing Batter, Nun chucks, Playing Piano, Pizza Tossing, Pole Vault, Pommel Horse, Pull Ups, Punch, Push Ups, Rock Climbing Indoor, Rope Climbing, Rowing, Salsa Spins, Skate Boarding, Skiing, Skijet, Soccer Juggling, Swing, Playing Tabla, TaiChi, Tennis Swing, Trampoline Jumping, Playing Violin, Volleyball Spiking, Walking with a dog, Yo Yo.

Các lệnh trong Tensorflow - Keras để xây dựng một mạng CNN 3D là tương tự như trường hợp 2D, tuy nhiên chúng ta sử dụng tích layer tích chập conv3D thay vì conv2D. Ví dụ các lệnh sau

```
conv2 = Conv3D(16, kernel_size=(3, 3, 3),padding='same')(conv1)
```

```
conv3 = LeakyReLU(alpha=.001)(conv2)
```

xây dựng các Layer kết hợp gồm:

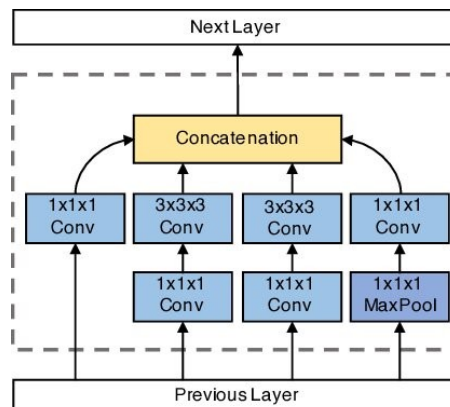
- 1) Layer convolution 3D với 16 filters, kích thước filter là 3x3x3, padding để đầu vào và đầu ra cùng kích thước. Layer này nhận đầu vào là conv2
- 2) Sử dụng activation là LeakyReLU $f(z) = \max\{x, \alpha \cdot x\}$ với $\alpha = -0.001$. Các version cao của keras.layers có thể không sử dụng tham số này, các bạn cần đổi tham số để tương thích.

Tương tự, layer MaxPool 3D có thể xây dựng như sau:

```
conv4 = MaxPooling3D(pool_size=(2, 2, 2), padding='same')(conv3)
```

Lệnh trên tạo Layer MaxPool 3D, kích thước 2x2x2, đầu vào là conv3 trước đó.

Đoạn lệnh dưới đây tạo thành 3D CNN, tuy nhiên để tránh mạng quá sâu - vì chúng ta cần huấn luyện mạng từ đầu nên nếu mạng quá sâu sẽ cần tính toán quá lớn trong bước lan truyền ngược – chúng ta sẽ xây dựng mạng theo chiều ngang tương tự kiểu block Inception:



Các khối nhân chập sẽ được padding sao cho kích thước giữ nguyên như đầu vào (sử dụng tham số “same”).

Đoạn lệnh xây dựng mạng này như sau:

```
input_x = Input(shape = (32, 32, depth, 3))

initial_conv = Conv3D(16, kernel_size= (3, 3, 3), padding='same')(input_x)
initial_conv = LeakyReLU(alpha=.001)(initial_conv)
```

```

initial_conv = Conv3D(32, kernel_size= (3, 3, 3), padding='same')(initial_conv)
initial_conv = LeakyReLU(alpha=.001)(initial_conv)

#####
# PARALLEL 1

conv1 = Conv3D(16, kernel_size=(1, 1, 1),padding='same')(initial_conv)
conv1 = LeakyReLU(alpha=.001)(conv1)
conv1 = MaxPooling3D(pool_size=(2, 2, 2), padding='same')(conv1)

conv1 = Conv3D(16, kernel_size=(1, 1, 1),padding='same')(conv1)
conv1 = LeakyReLU(alpha=.001)(conv1)
#####

conv1 = MaxPooling3D(pool_size=(2, 2, 2), padding='same')(conv1)

#####

#Parallel 2

conv2 = Conv3D(8, kernel_size=(1, 1, 1),padding='same')(initial_conv)
conv2 = LeakyReLU(alpha=.001)(conv2)
conv2 = MaxPooling3D(pool_size=(2, 2, 2), padding='same')(conv2)

#####

conv2 = Conv3D(16, kernel_size=(1, 1, 1),padding='same')(conv2)
conv2 = LeakyReLU(alpha=.001)(conv2)

conv2 = MaxPooling3D(pool_size=(2, 2, 2), padding='same')(conv2)

#####

#Parallel 3

conv3 = Conv3D(4, kernel_size=(1, 1, 1),padding='same')(initial_conv)
conv3 = LeakyReLU(alpha=.001)(conv3)
conv3 = MaxPooling3D(pool_size=(2, 2, 2), padding='same')(conv3)

#####

conv3 = Conv3D(16, kernel_size=(1, 1, 1),padding='same')(conv3)
conv3 = LeakyReLU(alpha=.001)(conv3)

conv3 = MaxPooling3D(pool_size=(2, 2, 2), padding='same')(conv3)

#####

added = keras.layers.Add()([conv1, conv2, conv3])

added = MaxPooling3D(pool_size=(2, 2, 2), padding='same')(added)

added = Flatten()(added)

dense_1 = Dense(256)(added)

dense_2 = Dense(2)(dense_1)

model = Model(input_x, dense_2)

model.compile(loss=categorical_crossentropy, optimizer=Adam(), metrics=['accuracy'])

model.summary()

```

Trong mạng trên:

- Các khối Parallel 1, 2, 3 là các khối nhân chập với các filter có kích thước 3x3x3 hoặc 1x1x1. Các đầu ra được đưa qua Activation LeakyReLU và MaxPool 3D.
- Sau khi nhân chập, các kết quả của các khối Parallel sẽ được cộng gộp lại, đưa qua MaxPool 3D và làm phẳng (Flatten)
- Vector kết quả làm phẳng sẽ được đưa qua một tầng Fully Connected để đưa về 256 chiều/ tầng tiếp theo đưa về 2 chiều và qua đầu ra để tính LOSS dạng CrossEntropy.

Chi tiết các Layer liên quan trong Tensorflow.Keras:

```
tf.keras.layers.Conv3D(
    filters,
    kernel_size,
    strides=(1, 1, 1),
    padding='valid',
    data_format=None,
    dilation_rate=(1, 1, 1),
    groups=1,
    activation=None,
    use_bias=True,
    kernel_initializer='glorot_uniform',
    bias_initializer='zeros',
    kernel_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    bias_constraint=None,
    **kwargs
)
```

Tham số:

filters	int, chiều của không gian đầu ra (số bộ lọc trong phép tích chập).
kernel_size	int hoặc tuple/danh sách gồm 3 số nguyên, chỉ định kích thước của cửa sổ tích chập.
strides	int hoặc tuple/danh sách gồm 3 số nguyên, chỉ định độ dài bước của phép tích chập. strides > 1 không tương thích với . dilation_rate > 1
padding	chuỗi, hoặc "valid" hoặc "same" (không phân biệt chữ hoa chữ thường). "valid" nghĩa là không có phần đệm. "same" dẫn đến phần đệm đều sang trái/phải hoặc lên/xuống của đầu vào. Khi padding="same" và strides=1, đầu ra có cùng kích thước với đầu vào.
data_format	chuỗi, hoặc 'channels_last' hoặc 'channels_first'. Thứ tự của các kích thước trong các đầu vào. tương ứng với các đầu vào có hình dạng trong khi tương ứng với các đầu vào có hình dạng . Giá trị mặc định được tìm thấy trong tệp cấu hình Keras của bạn tại . Nếu bạn không bao giờ đặt giá trị này, thì giá trị sẽ là . "channels_last" "channels_first" "channels_last" (batch_size, spatial_dim1, spatial_dim2, spatial_dim3, channels)"channels_first"(batch_size, channels, spatial_dim1, spatial_dim2, spatial_dim3)image_data_format~/keras/keras.json"channels_last"
dilation_rate	int hoặc tuple/danh sách gồm 3 số nguyên, chỉ định tỷ lệ giãn nở sử dụng cho phép tích chập giãn nở.
groups	Một số nguyên dương chỉ định số nhóm mà đầu vào được chia dọc theo trục kênh. Mỗi nhóm được tích chập riêng biệt với các bộ lọc. Đầu ra là sự nối tiếp của tất cả các kết quả dọc theo trục kênh. Đầu vào kênh và cả hai đều phải chia hết cho . filters // groupsgroupsfiltersgroups
activation	Chức năng kích hoạt. Nếu None, không có kích hoạt nào được áp dụng.
use_bias	bool, nếu True, độ lệch sẽ được thêm vào đầu ra.
kernel_initializer	Bộ khởi tạo cho hạt nhân tích chập. Nếu None, bộ khởi tạo mặc định () sẽ được sử dụng. "glorot_uniform"
bias_initializer	Bộ khởi tạo cho vector độ lệch. Nếu None, bộ khởi tạo mặc định ("zeros") sẽ được sử dụng.
kernel_regularizer	Bộ điều chỉnh tùy chọn cho hạt nhân tích chập.
bias_regularizer	Bộ điều chỉnh tùy chọn cho vector độ lệch.
activity_regularizer	Chức năng điều chỉnh tùy chọn cho đầu ra.
kernel_constraint	Hàm chiếu tùy chọn được áp dụng cho hạt nhân sau khi được cập nhật bởi một Optimizer(ví dụ được sử dụng để triển khai các ràng buộc chuẩn hoặc ràng buộc giá trị cho trọng số lớp). Hàm phải lấy biến không được chiếu làm đầu vào và phải trả về biến được chiếu (phải có cùng hình dạng). Không an toàn khi sử dụng các ràng buộc khi thực hiện đào tạo phân tán không đồng bộ.
bias_constraint	Hàm chiếu tùy chọn được áp dụng cho độ lệch sau khi được cập nhật bởi Optimizer.

Đầu ra: Kết quả tích chập 3D (tensor 5D activation(conv3d(inputs, kernel) + bias))

Layer MaxPool 3D

```
keras.layers.MaxPooling3D(
    pool_size=(2, 2, 2),
    strides=None,
    padding="valid",
    data_format=None,
    name=None,
    **kwargs
)
```

Tham số:

- **pool_size** : int hoặc tuple gồm 3 số nguyên, các hệ số để giảm tỷ lệ (dim1, dim2, dim3). Nếu chỉ định một số nguyên, cùng một độ dài cửa sổ sẽ được sử dụng cho tất cả các chiều.

- **strides** : int hoặc tuple gồm 3 số nguyên, hoặc None. Giá trị strides. Nếu None, mặc định sẽ là pool_size. Nếu chỉ có một int được chỉ định, cùng một kích thước stride sẽ được sử dụng cho tất cả các chiều.
- **padding** : chuỗi, "valid" hoặc "same" (không phân biệt chữ hoa chữ thường). "valid" nghĩa là không đệm. "same" dẫn đến việc đệm đều sang trái/phải hoặc lên/xuống của đầu vào sao cho nếu strides = 1x1x1 thì đầu ra có cùng chiều cao/chiều rộng với đầu vào. Chú ý: Trong code ví dụ, ta sử dụng padding = 'same', tuy nhiên phần strides chúng ta sử dụng mặc định nên strides = pool_size = 2x2x2. Do đó đầu ra kích thước mỗi chiều còn ½ kích thước đầu vào.
- **data_format** : chuỗi, "channels_last" hoặc "channels_first". Thứ tự của các kích thước trong các đầu vào. "channels_last" tương ứng với các đầu vào có hình dạng (batch, spatial_dim1, spatial_dim2, spatial_dim3, channels) trong khi "channels_first" tương ứng với các đầu vào có hình dạng (batch, channels, spatial_dim1, spatial_dim2, spatial_dim3). Giá trị mặc định image_data_format được tìm thấy trong tệp cấu hình Keras của bạn tại ~/.keras/keras.json. Nếu bạn không bao giờ đặt giá trị này, thì giá trị sẽ là "channels_last".

Bài tập:

- 1) Huấn luyện mô hình bằng khoảng 80% tập dữ liệu đã đề cập trong phần trước và chạy dự đoán cho 20% còn lại của tập dữ liệu. Đưa ra kết quả và đánh giá.
- 2) Sửa một số nhánh của Parallel 1, 2, 3 để sử dụng tích chập với filter 5x5x5. Chạy lại và đánh giá kết quả.
- 3) Thay vì xây dựng khối các tích chập theo chiều ngang (Parallel), hãy xây dựng kiến trúc gồm các tích chập nối trên theo kiểu nối tiếp (sử dụng Keras.Layers.Sequential như mạng CNN 2D đã học) và thực nghiệm lại.

B. 2D CNN & FUSION BASED MODEL

Code tham khảo có trong **Lab3_2DCNN_Based_Video_Classification.ipynb**

Dựa vào các bài thực hành đã có ở phần trước về các mạng CNN 2D (có thể sử dụng thư viện tùy ý – pytorch hoặc tensorflow.keras), cũng như kiến thức lý thuyết đã học về các mô hình kết hợp 2D CNN & Fusion, chúng ta có thể tự xây dựng được một mô hình dạng Fusion 2D CNN cho bài toán phân loại video, thực hiện các yêu cầu sau đây:

- Sử dụng bộ dữ liệu UCF101 hoặc UCF50 (download hoặc giáo viên cung cấp)
- **Xây dựng 01 kiến trúc 2D CNN** cho bài toán phân loại gồm: Một dãy tuần tự các Layer 2D CNN, lần lượt có các siêu tham số như sau:
 - Layer 1st: filter size: 3x3; số filter (số map đầu ra): 16; padding – default; activation: ReLU hoặc Leaky ReLU; stride – default.
 - MaxPool: Size 2x2
 - Dropout: rate = 0.3
 - Layer 2nd: filter size: 3x3; số filter (số map đầu ra): 32; padding – default; activation: ReLU hoặc Leaky ReLU; stride – default.
 - MaxPool: Size 2x2
 - Dropout: rate = 0.2
 - Layer 3rd: filter size: 3x3; số filter (số map đầu ra): 64; padding – default; activation: ReLU hoặc Leaky ReLU; stride – default.
 - MaxPool: Size 2x2
 - Dropout: rate = 0.1
 - Layer 4th: filter size: 3x3; số filter (số map đầu ra): 128; padding – default; activation: ReLU hoặc Leaky ReLU; stride – default.
 - MaxPool: Size 2x2
- **Xây dựng một kiến trúc phân loại dạng MLP gồm:**
 - Layer Flatten: Duỗi thẳng kết quả là các maps thu được từ phần CNN
 - Layer Fully Connected (Dense - MLP) 1st: Đưa kết quả là vector đã duỗi thẳng về 256 chiều – Activation: ReLU
 - Layer Fully Connected (Dense - MLP) 2nd: Đưa kết quả là vector 256 chiều về số chiều bằng số class (50 nếu dùng dữ liệu UCF50 hoặc 101 với dữ liệu UCF101) – Activation: SOFTMAX.
 - Thiết lập để sử dụng Loss Function là Category-CrossEntropy.
- Kết hợp với Fusion để hình thành 02 mô hình dưới đây:

(a) Mô hình 2DCNN với Late Fusion

- Bổ sung tham số đầu vào với Layer 1st phần CNN: input size đưa về một cỡ ảnh phù hợp (ví dụ 150x150).
- Bổ sung công cụ phân rã thành các frames (tham khảo tệp code video2frames.ipynb đính kèm). Từ đó xây dựng chức năng kiểm tra số frame có trong video đầu vào (giả sử là T) và phân rã chúng.

- Bổ sung 1 Layer Input, trong đó số channel đầu vào là 3T, thực hiện tuần tự theo kiến trúc nhân chập ở trên (kết quả các map cho mỗi frame cần được giữ đầy đủ)
- Bổ sung Layer ở tầng tích chập cuối, sử dụng một bộ tích chập 1x1 để cộng gộp các maps / hoặc sử dụng Layer Concatenate để ghép các kết quả lại.
- Đưa qua phần phân loại.

(b) Mô hình 2DCNN với Early Fusion

- Bổ sung tham số đầu vào với Layer 1st phần CNN: input size đưa về một cỡ ảnh phù hợp (ví dụ 150x150), số channel đầu vào là 3T, đầu ra đưa về 01 map.
- Đưa qua phần phân loại.

Thực nghiệm và so sánh kết quả của hai cấu hình EARLY và LATE đề cập đến ở trên.

PHÂN LOẠI HÀNH ĐỘNG TRONG VIDEO BẰNG 2D CNN KẾT HỢP FUSION SỬ DỤNG PYTORCH

Trong file nguồn tham khảo **UCF50_2D_CNN_Fusion_PyTorch.ipynb** là chương trình tương, phân loại video dựa trên 2D CNN và Fusion, nhưng được viết trên python và Torch. Các bạn tham khảo chương trình nguồn, một số tùy chọn cho cấu hình như sau:

Cách dùng nhanh

- (I) Mở notebook và chỉnh `cfg.DATA_ROOT` cho đúng đường dẫn UCF50 (ví dụ Kaggle: `/kaggle/input/ucf50/UCF50` hoặc đường dẫn local của bạn).
- (II) Chọn kiểu fusion trong `cfg.FUSION`:
 - a. "late" (mặc định, trộn logits 2 nhánh)
 - b. "early_feature" (nối đặc trưng trước classifier)
 - c. "early_channel" (ghép RGB + flow theo kênh, sửa conv1)
- (III) Tùy tài nguyên máy, tối ưu:
 - a. Giảm `FLOW_STACK` (ví dụ 3), `IMG_SIZE` (168/128)
 - b. Chọn `FLOW_METHOD="farneback"` cho nhanh
 - c. Bật cache `FLOW_PRECOMPUTE=True` để tính optical flow một lần và dùng lại
- (IV) Chạy tuần tự các cell "Train", sau đó phần "Evaluation" sẽ in classification report + vẽ **confusion matrix**. Mô hình tốt nhất được lưu ở `outputs_ucf50_fusion/best_model.pth`.
- (V) Dùng cell "Inference" để dự đoán nhanh trên một video .avi.

Mô tả chi tiết cho mỗi đoạn lệnh đã có trong chương trình nguồn. Các bạn cần chỉnh lại cấu hình đường dẫn tới dataset để thực hiện thực nghiệm.

Bài tập: Mô hình 2D CNN được sử dụng trong đoạn code trên là **resnet18**, hãy thay nó bằng một mô hình khác hiệu quả hơn, ví dụ EfficientNet hoặc nhẹ hơn hẳn, ví dụ MobileNet V3. Thực hiện lại thực nghiệm và đánh giá kết quả.

C. TWO STREAM MODELS

Chương trình trong file notebook **two_stream_ucf50_videos_classifier.ipynb - Two-Stream (RGB + Optical Flow) cho UCF50** là chương trình phân loại hành động trong video bằng thuật toán Two- Streams, gồm RGB Stream và Optical Flow Stream, xây dựng bằng PyTorch.:

Nội dung công việc chính của chương trình

- Cấu hình linh hoạt (đường dẫn `data_root`, số epoch, batch size, số segment thời gian, số khung flow, chọn TV-L1 hoặc Farneback, v.v.).
- Dataset loader đọc **video từ thư mục mỗi lớp** (UCF50 layout), **sample khung theo kiểu TSN**, resize + center-crop, và **tính optical flow on-the-fly** bằng OpenCV.
- Mô hình **Two-Stream**:
 - Backbone ResNet (18/34/50) cho RGB.
 - Backbone ResNet với conv đầu vào được sửa cho **C = 2 × flow_stack** (stack (u, v)).

- **Fusion:** late_sum (trọng số) hoặc concat_fc (ghép đặc trưng rồi FC).
- Train/Val split tự động theo tỉ lệ, **checkpoint** mỗi epoch + **best_model.pth**, lưu config.json, class_to_idx.json, history.json, metrics_log.json.
- **Vẽ đồ thị** loss/accuracy và (tùy chọn) **confusion matrix** + classification_report.txt.
- Hàm **inference** cho 1 video mẫu.

Cách dùng nhanh

1. Mở notebook và chỉnh trong ô "Configuration":
 - data_root = "/path/to/UCF50" (thư mục: UCF50/ClassName/*.avi).
 - (Tùy chọn) giảm num_segments, flow_stack hoặc đặt flow_method="farneback" nếu máy yếu.
2. Chạy lần lượt các cell: build dataset → init model → train → plot → (optional) confusion matrix.
3. Kết quả (metric, hình vẽ, checkpoints) sẽ ở ./two_stream_runs/ucf50_exp1/ (có thể đổi output_dir).

Gợi ý tối ưu/biến thể

- Muốn nhanh hơn: num_segments=2, flow_stack=5, flow_method="farneback", batch_size nhỏ, backbone resnet18.
- Muốn chính xác hơn: resnet50, concat_fc, tăng num_segments (nhưng sẽ chậm).
- Có thể **tiền tính flow** và cache .npy để tăng tốc; notebook đã tổ chức mã để bạn dễ thay phần tính flow bằng "load từ đĩa".

Ngoài ra, trong chương trình cũng có một số phần công việc gồm

- **Tiền xử lý & cache Optical Flow ra đĩa**
 - Tùy chọn use_flow_cache, flow_cache_dir, precompute_flow_now, precompute_on_split, cache_flow_method.
 - Hàm tiền xử lý: đọc video → resize+center-crop như lúc train → tính **pairwise flow** giữa khung t→t+1 → scale/clip → lưu **một file .npy**/video (shape (N-1, H, W, 2)).
 - DataLoader phiên bản mới sẽ **ưu tiên lấy flow từ cache** để ghép **flow stack** quanh frame trung tâm (vẫn giữ đúng logic Two-Stream/TSN). Nếu không có cache sẽ fallback tính on-the-fly và có thể tự cache.
 - Có **index.json** trong flow_cache_dir để map video_path → file cache.
- **LR Scheduler**
 - Chọn lr_scheduler_type trong ["cosine", "step", "plateau", "none"].
 - Tham số: step_size, gamma, cosine_Tmax.
 - Lồng vào vòng train; đối với plateau sẽ step theo metric được theo dõi.
- **Early Stopping**
 - Theo dõi early_stop_on ("val_acc" hoặc "val_loss") với early_stop_mode ("max"/"min") và early_stop_patience.
 - Tự dừng khi không cải thiện đủ số epoch liên tiếp; **best_model.pth** được cập nhật khi có cải thiện.
- **Xuất mô hình**
 - Lưu **state_dict**: two_stream_state_dict.pt.
 - Xuất **TorchScript**:
 - Thử torch.jit.script(model); nếu không tương thích thì **fallback** sang trace với dummy input cùng kích thước cấu hình.
 - File xuất: two_stream_scripted.pt hoặc two_stream_traced.pt.
 - Tất cả nằm trong export_dir.

Cách bật precompute/caching nhanh

- Mở ô “Extended Configuration”:
 - `use_flow_cache = True`
 - `flow_cache_dir = "./two_stream_runs/flow_cache"`
 - `precompute_flow_now = True` và `precompute_on_split = "train+val"`
- Chạy ô “Flow Precompute & Caching Utilities” để tạo cache trước khi train.

Gợi ý thực tế

- Máy yếu: dùng `farneback`, `num_segments=2`, `flow_stack=5`, backbone `resnet18`.
- Máy khỏe: `tv11`, tăng `num_segments`, thử `concat_fc` hoặc `resnet50`.
- Khi triển khai: dùng **TorchScript** (scripted ưu tiên, traced dự phòng) để inference nhanh và độc lập Python source.

Trong chương trình đã có giải thích chi tiết cho từng đoạn lệnh. Các bạn có thể bám theo đó để thực hiện.

D. 2D CNN & RNN BASED MODEL

Code tham khảo có trong `VideoClass_CNNRNN..ipynb`

Như trên đã phân tích, chúng ta sẽ dựa vào một kiến trúc 2D CNN để trích xuất đặc trưng của các Frame, sau đó đưa vector feature thu được làm đầu vào của một mạng RNN, theo thứ tự chuỗi.

Mạng RNN có dạng Many – One, với 01 đầu ra là kết quả dự đoán phân loại của chuỗi đầu vào.

Dựa vào các bài thực hành đã có ở phần trước về các mạng CNN 2D (có thể sử dụng thư viện tùy ý – pytorch hoặc tensorflow.keras), cũng như kiến thức lý thuyết đã học về các mô hình RNN, chúng ta có thể tự xây dựng được một mô hình dạng 2D CNN - LSTM cho bài toán phân loại video.

Chi tiết về mô hình các bạn tham khảo trong tệp nguồn chương trình.: Mô hình CNN 2D được sử dụng ở đây là ResNet152. Các bạn hãy thay nó bằng một mô hình hiệu quả hơn.